Phase 1 Implementation Plan: Adoption of the Quartet Filing Doctrine and Repository Restructuring

1. Executive Summary and Strategic Context

The maturation of the **Stela** repository architecture necessitates a fundamental and rigorous paradigmatic shift in the categorization, storage, and lifecycle management of its ephemeral and persistent state artifacts. The existing governance model, as currently codified in the *Policy of Truth* (PoT), relies on a binary classification system where the storage/ directory functions as a monolithic "Trash" receptacle.[1] While this designation successfully established a boundary between canonical source code and non-canonical runtime artifacts, it lacks the semantic granularity required for high-fidelity automated operations, advanced observability, and long-term provenance preservation. As the system evolves toward an "Immutable Workflow" and a "One Truth" context engine, the conflation of transportable state, process continuity data, observability signals, and historical records under a single "Trash" label creates unacceptable risks of context contamination, data loss, and operational ambiguity.

This report presents the comprehensive **Phase 1 Implementation Plan** for the strict enforcement of the **Quartet Filing Doctrine**. This new doctrine establishes a sophisticated taxonomy for non-canonical data, aligning the repository's structure with industrial best practices observed in high-reliability systems, such as those governed by NASA's payload telemetry standards and enterprise-grade cloud architectures.[2] The central objective is the migration from the legacy "Trash" hierarchy to four distinct, semantically rigorous root locations: storage/ (redefined as **Payload**), var/tmp/ (designated as **Resume**), logs/ (designated as **Telemetry**), and archives/ (designated as **Cold** storage).

This restructuring is not merely a cosmetic reorganization; it is a functional imperative. The separation of "Resume" state—volatile, machine-local data required for process continuity—from "Payload" state—transportable, inter-agent data—eliminates the critical risk of accidental context contamination, where a local cache file might be erroneously ingested as a canonical handoff artifact. Furthermore, the elevation of archives/ to a root-level directory underscores the compliance value of the *Proof of Work* (PoW) and *State of Play* (SoP) ledgers, removing them from the "Trash" hierarchy and shielding them from aggressive hygiene logic intended for disposable temporary files.[1] Similarly, the creation of a dedicated logs/ root allows for the integration of standardized telemetry ingestion pipelines without polluting the working tree or risking the leakage of sensitive data into state dumps.[4]

The following sections detail the precise architectural logic, forensic analysis, tooling refactoring requirements, and migration protocols necessary to execute this transition. This

plan serves as the authoritative blueprint for modifying the Ops Kernel—specifically ops/bin/prune, ops/bin/dump, and tools/verify.sh—and amending the *Policy of Truth* to codify these new physical laws of the repository.

2. Theoretical Framework and The Semantic Gap

To understand the necessity of the Quartet Filing Doctrine, one must first analyze the deficiencies of the current "Storage as Trash" model through the lens of systems theory and information lifecycle management.

2.1 The "Trash" Semantics vs. State Value

In the current *Policy of Truth*, storage/ is defined as "Trash (local artifacts, never canon)".[1] This definition implies that the contents of storage/ are valueless and disposable. However, a forensic examination of the artifacts actually stored there reveals a contradiction. The storage/handoff/ directory contains OPEN and RESULTS artifacts, which constitute the **critical path** for inter-agent collaboration. If an OPEN artifact is deleted, the agent loses its session context. If a RESULTS artifact is deleted before closeout, the proof of work is lost.[1]

Therefore, treating high-value transport packets as "Trash" is a semantic error. It invites aggressive cleanup policies that endanger active workflows. By redefining storage/ as **Payload**, we align the directory's name with its function: the storage of valuable, serialized state packages that are in transit between entities.

2.2 The "Tmp" Ambiguity and Resume Semantics

The current storage/tmp/ directory is used for volatile scratch space. However, in distributed systems and long-running agent workflows, there is a crucial distinction between "ephemeral scratch" (which can be deleted at any time) and "continuity state" (which must survive a process restart).

The Unix filesystem standard differentiates between /tmp (cleared on reboot) and /var/tmp (preserved across reboots).[5] In the context of the Stela repository, an agent session might be interrupted by a network failure or a preemption event. To resume strictly from where it left off, the agent requires a "Resume" state—lock files, partial synthesis buffers, or intermediate calculations—that survives the interruption. The legacy storage/tmp/ location, buried under "Trash," suggests volatility. Migrating to var/tmp/ at the repository root explicitly invokes the /var/tmp semantic of **persistence-during-interruption**, signaling to both human operators and automated hygiene scripts that these files should only be scrubbed when a session is definitively concluded, not merely because they are "temporary."

2.3 Telemetry Isolation and The Observer Effect

Currently, logs and debug outputs are likely intermingled within storage/ or scattered across

the working tree. This violates the principle of **Telemetry Isolation**. Observability data (logs, metrics, traces) behaves differently from payload data. Payloads are input/output; Telemetry is side-effect data.

Mixing telemetry with payloads creates two risks:

1. **Context Pollution:** If an agent ingests a directory to understand the current state (e.g., via ops/bin/dump), including verbose logs can overflow the context window with irrelevant noise.
2. **Data Leakage:** Telemetry often contains PII, tokens, or raw request bodies that should not be serialized into a dump.[4]

By establishing logs/ as a root directory, we enable rigorous .gitignore policies and distinct lifecycle management. We can configure the dump tool to categorically exclude logs/, ensuring that the "Observer" (telemetry) does not interfere with the "Observed" (payload).

2.4 Cold Storage and The Archive Imperative

The *Proof of Work* (PoW) ledger is the repository's legal record of execution. Currently, older entries are archived to storage/archives/root/.[1] Placing these archives under "Trash" diminishes their perceived importance. They are not trash; they are the repository's long-term memory.

The **Cold** storage designation, implemented via the archives/ root, aligns with the "Archive Program" philosophy seen in major code vaults.[8] Cold storage implies immutability and long-term retention. By elevating archives to the root, we signal that this data is a permanent fixture of the repository's history, distinct from the active "hot" data in storage/. This separation facilitates compliance audits, where an auditor can simply review archives/ to see the complete history of operations without navigating through active operational debris.

3. Repository Forensics and Current State Audit

Before any migration command is executed, we must perform a rigorous forensic audit of the existing repository state to identify all dependencies, hardcoded paths, and implicit assumptions that will be broken by the restructuring. This audit relies on the provided dump manifest and file contents.[1]

3.1 Directory Topology Analysis

The current php-nuke-ce repository operates under the legacy Filing Doctrine. The storage/ directory is the gravitational center for all generated artifacts.

- **storage/ (The Legacy Root):**
  - **storage/archives/:** This directory is the primary target for the ops/bin/prune tool. It currently holds root/ (for PoW/SoP overflows), and likely tasks/, skills/, and agents/ for

harvested drafts.[1]
    - **storage/tmp/:** Used for volatile operations. The snippet for tools/verify.sh explicitly checks for the existence of storage/tmp, marking it as a critical infrastructure component.[1]
    - **storage/handoff/:** Contains OPEN artifacts (e.g., OPEN-work-dp-ops-0065...txt) and RESULTS artifacts. These are the lifeblood of the agent workflow.
    - **storage/dumps/:** Contains the output of ops/bin/dump.
    - **storage/dp/:** Contains active/allowlist.txt and intake/ for Dispatch Packets.

3.2 Dependency Map and Code Touchpoints

The migration will disrupt several key binaries. We have identified specific files that contain hardcoded references to the legacy paths:

1. **ops/bin/prune:**
    - *Evidence:* The snippet [1] shows the script source. It defines ARCHIVE_DIR="${REPO_ROOT}/storage/archives/root" and TMP_DIR="${REPO_ROOT}/storage/tmp".
    - *Impact:* If we move storage/archives to archives/, prune will fail to find old logs to rotate, or worse, it will recreate storage/archives and split the history. If we remove storage/tmp, the clean_storage_tmp function will fail or do nothing.
    - *Requirement:* prune must be refactored to point to archives/root and var/tmp.
2. **ops/bin/dump:**
    - *Evidence:* The dump manifest [2] shows that storage/.gitignore is included. ops/bin/dump likely uses git ls-files or similar logic.
    - *Impact:* The dump tool defines the "Payload". We must ensure that dump continues to capture storage/ (now strictly Payload) but *excludes* logs/ (Telemetry) and var/tmp/ (Resume) by default, while potentially offering an option to include archives/ (Cold) if a full historical dump is requested.
3. **tools/verify.sh:**
    - *Evidence:* The snippet [1] reveals that verify.sh contains a check: if [[! -d "storage/tmp"]]; then fail....
    - *Impact:* This script is a "Repo Gate".[1] If we delete storage/tmp without updating this script, the CI pipeline (GitHub Actions) will fail immediately, blocking all work.
    - *Requirement:* The verification logic must be updated to mandate var/tmp, logs, and archives while removing the mandate for storage/tmp.
4. **ops/bin/context:**
    - *Evidence:* While the full source isn't in the snippet, context tools typically use mktemp. If mktemp is configured to use storage/tmp, it must be redirected to var/tmp.
5. **Lifecycle Scripts (agent.sh, task.sh, skill.sh):**
    - *Evidence:* The snippet implies these scripts harvest drafts. These drafts currently go to storage/archives.

- *Requirement:* These scripts must be updated to save drafts to archives/agents, archives/tasks, and archives/skills.

3.3 The Risk of "Split-Brain" Ledgers

A critical risk identified during the audit involves the SoP and PoW ledgers. If the migration is performed while an agent is active, or if prune is not updated simultaneously, we risk a "split-brain" scenario where new archival entries are written to a new location while old entries remain in the old location. This would break the continuity of the provenance chain.

Mitigation: The migration must be atomic. The directory move and the code update must happen in the same Pull Request (PR). The system must be quiescent (no active DPs) during the transition.

4. The New Quartet Filing Doctrine: Definitions and Invariants

This section provides the formal specification for the new Filing Doctrine. These definitions will be codified in the *Policy of Truth* (PoT) and enforced by the Ops Kernel.

4.1 storage/ as Payload

Definition: The storage/ directory is the designated port for **Payloads**. A Payload is defined as any serialized artifact intended for transport between entities (Operator ↔ System ↔ Agent). It is the "Outbox" and "Inbox" of the repository.

Invariants:

- **Transportability:** Content must be self-contained and moveable.
- **Handoffs:** Must contain handoff/ for OPEN (start state) and RESULTS (end state) artifacts.
- **Snapshots:** Must contain dumps/ for repository state serialization.
- **Dispatch:** Must contain dp/ for Dispatch Packet processing.
- **No Volatility:** Must *not* contain temporary scratch files or locks (moved to var/tmp).
- **No History:** Must *not* contain archived logs (moved to archives).

Rationale: By restricting storage/ to Payloads, we allow for aggressive caching and transmission strategies. We can safely "tar up storage/" and ship it to an agent, knowing it contains exactly what is needed for the job and nothing else.

4.2 var/tmp/ as Resume

Definition: The var/tmp/ directory is the designated location for **Resume** state. These are non-canonical, local artifacts required to maintain session continuity, manage concurrency, or

recover from process interruption.

Invariants:

- **Local-Only:** Must be strictly ignored by git (.gitignore).
- **Persistence:** Must be resilient to process termination. Logic using this directory must expect files to exist from previous runs (unlike system /tmp).
- **Hygiene:** Safe to scrub only when no sessions are active.
- **Usage:** Lock files (.lock), temporary synthesis buffers, intermediate calculation caches, partial downloads.

Rationale: Adopting the /var/tmp standard [5] clarifies the lifecycle of these files. They are "temporary" but "valuable" for the duration of a session. This distinction prevents the "blind cleaning" that often plagues generic tmp directories.

4.3 logs/ as Telemetry

Definition: The logs/ directory is the designated sink for **Telemetry**. This includes structured logs, debug streams, error traces, and metric outputs generated by the Ops Kernel or the payload application.

Invariants:

- **Write-Only:** Processes should append to logs, not read them for state (state belongs in storage/ or var/tmp/).
- **Isolation:** Must be physically separate from Payloads to prevent PII/secret leakage.[7]
- **Rotation:** Subject to aggressive rotation and compression policies, independent of the repository's semantic state.

Rationale: Centralizing telemetry in logs/ simplifies the configuration of external observability tools (e.g., Filebeat, OpenTelemetry Collectors).[9] It allows for a single, comprehensive .gitignore rule for all log data, reducing the risk of accidental commits.

4.4 archives/ as Cold Storage

Definition: The archives/ directory is the designated vault for **Cold** storage. This contains historical records that have passed their active lifecycle but must be retained for compliance, audit, provenance, or disaster recovery.

Invariants:

- **Root-Level:** Exists at the top level to signify permanence.[8]
- **Immutability:** Once written, archives should rarely change (append-only ledgers or read-only drafts).
- **Organization:** Must be structured by domain (root/ for ledgers, tasks/ for drafts, etc.).
- **Compliance:** Serves as the primary source for "Negative Proof" (evidence of what was

tried and failed) and historical lineage.

Rationale: Removing archives from "Trash" elevates the status of history. It aligns with the "Cold Storage" concept [3], where access frequency is low but durability requirements are high.

5. Tooling Architecture and Refactoring Plan

The migration of directories is the easy part; the challenge lies in refactoring the Ops Kernel to respect the new topology. We must modify ops/bin/prune, ops/bin/dump, tools/verify.sh, and the lifecycle scripts.

5.1 Refactoring ops/bin/prune

The prune binary is the hygiene enforcer. Based on the source code in [1], it has hardcoded paths that must be changed.

Current State (Analysis of [1]):

Bash

```
ARCHIVE_DIR="${REPO_ROOT}/storage/archives/root"
TMP_DIR="${REPO_ROOT}/storage/tmp"
...
clean_storage_tmp() {
  if]; then return 0; fi
  ...
}
```

Refactoring Logic:

1. **Update Variables:** Change ARCHIVE_DIR to point to archives/root. Change TMP_DIR to point to var/tmp.
2. **Rename Functions:** Rename clean_storage_tmp to clean_resume_cache to reflect the new "Resume" semantic.
3. **Update Verification:** Ensure prune checks for the existence of the new directories before attempting operations.

Proposed Code:

Bash

```
# ops/bin/prune refactor
ARCHIVE_DIR="${REPO_ROOT}/archives/root"
RESUME_DIR="${REPO_ROOT}/var/tmp"  # Renamed from TMP_DIR

#...

clean_resume_cache() {  # Renamed from clean_storage_tmp
  if]; then return 0; fi
  #... logic to scrub contents...
  echo "Resume cache (var/tmp) scrubbed."
}
```

5.2 Refactoring tools/verify.sh

This script acts as a gatekeeper. If it fails, the repo is considered "broken."

Current State (Analysis of [1]):

Bash

```
# 2. Storage Hygiene Check
if [[! -d "storage/tmp" ]]; then
  fail "Missing required storage: 'storage/tmp/'"
fi
```

Refactoring Logic:

1. **Remove Legacy Check:** Delete the block checking for storage/tmp.
2. **Add Quartet Checks:** Add blocks to verify var/tmp, logs, and archives.
3. **Update Filng Doctrine Checks:** Ensure the script enforces that logs/ contains only log files (if possible) or at least exists.

Proposed Code:

Bash

```
# tools/verify.sh refactor

# 1. Platform Skeleton Check
required_dirs=(
    "ops" "docs" "opt" "tools" "projects" ".github"
    "logs" "archives" "var"  # Added new roots
)

#...

# 2. Storage Hygiene Check
# Removed storage/tmp check

# 3. Resume Hygiene Check
if [[! -d "var/tmp" ]]; then
    fail "Missing required Resume directory: 'var/tmp/'"
fi
```

5.3 Refactoring ops/lib/scripts/*.sh (Lifecycle Scripts)

The scripts agent.sh, task.sh, and skill.sh harvest drafts. They define a HANDOFF_DIR or DRAFT_DIR pointing to storage/archives/....

Refactoring Logic:

1. **Identify Variables:** Grep for storage/archives in ops/lib/scripts/.
2. **Update Paths:**
    - agent.sh: storage/archives/agents -> archives/agents
    - task.sh: storage/archives/tasks -> archives/tasks
    - skill.sh: storage/archives/skills -> archives/skills

Rationale: This ensures that newly harvested drafts are immediately placed in Cold storage (archives) rather than the deprecated storage/ path.

5.4 Refactoring ops/bin/open and ops/bin/dump

While dump targets storage/dumps (which remains valid under "Payload"), we must ensure it doesn't accidentally dump the new root directories if they are not intended for transport.

Refactoring Logic:

1. **Review dump Exclusions:** Ensure dump logic excludes logs/, var/, and archives/ unless explicitly requested. The legacy dump likely relied on .gitignore. Since we are updating .gitignore (see Section 6), dump might inherit these exclusions automatically.
2. **Review open Output:** open writes to storage/handoff. This is correct under the Payload

definition. No change needed for the *target*, but the *content* of the OPEN artifact (the prompt) should be updated to reference the new paths if it lists file locations for the agent.

6. Migration Protocol: Step-by-Step Execution

This section outlines the specific commands to execute the migration. This protocol is designed to be atomic and reversible up until the final deletions.

6.1 Step 1: Root Topology Initialization

Objective: Establish the new directory skeleton and secure it with git hygiene.

Action A: Create Directories

We must create the new root directories. storage/ already exists, so we focus on the new three.

Bash

```
mkdir -p var/tmp
mkdir -p logs
mkdir -p archives/root
mkdir -p archives/tasks
mkdir -p archives/skills
mkdir -p archives/agents
mkdir -p archives/context
```

Action B: Configure Git Ignore We must immediately secure these new directories. We will update the root .gitignore. The current .gitignore ignores storage/tmp/.[1] We need to add rules for the new roots.

Update .gitignore:

Code snippet

```
#... existing ignores...
```

```
# NEW DOCTRINE: Resume
var/tmp/
!var/tmp/.gitkeep

# NEW DOCTRINE: Telemetry
logs/
!logs/.gitkeep

# NEW DOCTRINE: Cold Storage
# Archives structure is tracked, contents are typically local/drafts unless promoted.
archives/*
!archives/root/
!archives/tasks/
!archives/skills/
!archives/agents/
!archives/context/
!archives/**/.gitkeep
```

Action C: Place Anchors

Placing .gitkeep files ensures empty directories are tracked by git, preserving the skeleton for other clones.

Bash

```
touch var/tmp/.gitkeep
touch logs/.gitkeep
touch archives/root/.gitkeep
#... (and for other subdirs)
```

6.2 Step 2: Migration of Archives (Cold)

Objective: Move existing ledger archives and draft artifacts from storage/archives to archives/.

Command Protocol:

Bash

```
# Move Ledger Archives (PoW, SoP)
# Check if source exists to avoid errors
if [ -d "storage/archives/root" ]; then
    echo "Migrating Ledger Archives..."
    mv storage/archives/root/* archives/root/ 2>/dev/null |

| true
fi

# Move Task Drafts
if [ -d "storage/archives/tasks" ]; then
    echo "Migrating Task Drafts..."
    mv storage/archives/tasks/* archives/tasks/ 2>/dev/null |

| true
fi

#... Repeat for skills, agents, context...
```

6.3 Step 3: Migration of Tmp (Resume)

Objective: Clear the old scratch space and initialize the new Resume space.

Command Protocol:

Contents of storage/tmp are volatile. We do not migrate them; we purge them.

Bash

```
# Clean old tmp
echo "Purging legacy scratch space..."
rm -rf storage/tmp/*
```

6.4 Step 4: Redefinition of Storage (Payload)

Objective: Clean up storage/ to reflect its new, narrower definition.

Command Protocol:

Bash

```
# Remove deprecated directory structures
rmdir storage/archives/root
rmdir storage/archives/tasks
#... remove other subdirs...
rmdir storage/archives 2>/dev/null |

| true
rmdir storage/tmp 2>/dev/null |

| true
```

Result: storage/ now contains only handoff/, dumps/, dp/, and root meta-files.

7. Governance Updates: Amending the Constitution

The *Policy of Truth* (PoT) is the final authority. Code changes without PoT updates constitute "Canon Drift".[1] We must amend Section 1.1 to reflect the Quartet Doctrine.

7.1 PoT.md Amendment

Current Text:

- storage/ = Trash (local artifacts, never canon).

New Text (to be applied):

1.1 Filing Doctrine

Filing:

- ops/ = Run (binaries, manifests, automation).
- docs/ = Explain (manuals and rationale).
- opt/ = Isolate (JIT-only content excluded from global context).
- projects/ = Work (payload code).
- storage/ = **Payload** (Transported state: handoffs, dumps, DPs).
- var/tmp/ = **Resume** (Process state and continuity caches).
- logs/ = **Telemetry** (Observability signals).
- archives/ = **Cold** (Historical records and compliance vaults).

7.2 docs/MANUAL.md Update

The Manual must be updated to guide operators to the new paths.

- **Artifacts Section:** Update references from storage/archives to archives/.
- **Prune Command:** Update documentation to explain that prune now cleans var/tmp (Resume) and rotates ledgers to archives/ (Cold).

8. Risk Assessment and Mitigation Strategies

Risk	Probability	Impact	Mitigation Strategy
Data Loss (Archives)	Low	High	Use mv (move) rather than cp + rm. Verify file counts in archives/root match storage/archives/root before deleting the source directory.
Tooling Breakage	High	Medium	The "Expand and Contract" pattern (create new dirs first) allows us to fix tooling incrementally. The update to tools/verify.sh is the critical path; it must be done *before* deleting storage/tmp.
Context Drift	Medium	Low	Agents might have cached paths. Force a "Freshness Gate" restart (./ops/bin/open) after migration to reload the new

			context.
Git Noise	High	Low	Moving archives will generate delete/add events in git if they were tracked. This is acceptable for a structural refactor. Use git mv if files were tracked to preserve history.

9. Conclusion

The transition to the **Quartet Filing Doctrine** represents a critical maturation of the Stela repository's underlying philosophy. We are moving from a simplified model where all runtime artifacts were treated as "Trash" to a sophisticated, semantic architecture that recognizes the distinct value and lifecycle of **Payloads** (transport), **Resume** state (continuity), **Telemetry** (observability), and **Cold** storage (history).

This plan provides a rigorously audited, step-by-step path to execution. By aligning the directory structure with the functional realities of the system—and with industrial standards for high-reliability software—we lay the groundwork for more advanced automation, secure telemetry pipelines, and unshakeable provenance tracking. The Ops Kernel refactoring ensures that these new "physical laws" are not just documented in the PoT, but enforced by the code itself.

Detailed Implementation Checklist (Copy/Paste for Operator)

- [ ] **1. Safety Backup**
  - [ ] git status (Ensure clean tree).
  - [ ] ./ops/bin/dump --scope=full --out=backup-pre-quartet.tar.xz (Preserve pre-migration state).
- [ ] **2. Root Topology Initialization**
  - [ ] mkdir -p var/tmp logs archives/{root,tasks,skills,agents,context}
  - [ ] touch var/tmp/.gitkeep logs/.gitkeep archives/root/.gitkeep
- [ ] **3. Git Configuration**
  - [ ] Edit .gitignore: Add var/tmp/, logs/, archives/* rules. Remove storage/tmp/ rule.
- [ ] **4. Tooling Refactor (Code Updates)**

    - [ ] **tools/verify.sh**: Replace storage/tmp check with var/tmp, logs, and archives.
    - [ ] **ops/bin/prune**: Update ARCHIVE_DIR to archives/root, TMP_DIR to var/tmp.
    - [ ] **ops/bin/context**: Update temp file path to var/tmp.
    - [ ] **ops/lib/scripts/*.sh**: Update HANDOFF_DIR/DRAFT_DIR to archives/*.
- [ ] **5. Data Migration**
    - [ ] mv storage/archives/root/* archives/root/ (if exists).
    - [ ] mv storage/archives/tasks/* archives/tasks/ (if exists).
    - [ ] mv storage/archives/skills/* archives/skills/ (if exists).
    - [ ] mv storage/archives/agents/* archives/agents/ (if exists).
    - [ ] rm -rf storage/tmp/* (Purge legacy resume state).
- [ ] **6. Cleanup**
    - [ ] rmdir storage/archives/* (Remove subdirs).
    - [ ] rmdir storage/archives (Remove root archive dir).
    - [ ] rmdir storage/tmp (Remove root tmp dir).
- [ ] **7. Governance Updates**
    - [ ] Edit **PoT.md**: Update Section 1.1 with Quartet definitions.
    - [ ] Edit **docs/MANUAL.md**: Update paths in reference sections.
- [ ] **8. Verification & Closeout**
    - [ ] ./tools/verify.sh (Must PASS).
    - [ ] ./ops/bin/prune --dry-run (Must target new paths).
    - [ ] Commit changes: git commit -am "DP-OPS-0068: Enforce Quartet Filing Doctrine (Payload/Resume/Telemetry/Cold)".

Works cited

1. dump-full-main-80bb3afd.txt
2. Best Practices for Hosted Payload Interface Design Guidelines - NASA Science, accessed February 16, 2026, https://science.nasa.gov/wp-content/uploads/2023/06/Best-Practices-essp2.pdf
3. AWS Well-Architected Framework, accessed February 16, 2026, https://docs.aws.amazon.com/pdfs/wellarchitected/latest/framework/wellarchitected-framework.pdf
4. Scrubbing Sensitive Data from OpenTelemetry Logs, Traces & Metrics - Dash0, accessed February 16, 2026, https://www.dash0.com/guides/scrubbing-sensitive-data-with-opentelemetry
5. [CSF] Enable Continuous Delivery of Your Resume With GitHub Actions - DZone, accessed February 16, 2026, https://dzone.com/articles/enable-continuous-delivery-of-your-resume-with-git-1
6. refuse systemd-tmpfiles --purge invocation without config file specified on cmdline #33349, accessed February 16, 2026, https://github.com/systemd/systemd/issues/33349
7. Keep PII Out of Your Telemetry: Sanitizing Logs, Traces, and Metrics with OpenTelemetry, accessed February 16, 2026,

https://oneuptime.com/blog/post/2025-11-13-keep-pii-out-of-observability-telemetry/view
8. FAQs - GitHub Archive Program, accessed February 16, 2026, https://archiveprogram.github.com/faq/
9. Mastering OpenTelemetry: A Practical Guide to Metrics, Logs and Tracing in .Net - Medium, accessed February 16, 2026, https://medium.com/@caner.demirci/mastering-opentelemetry-a-practical-guide-to-metrics-logs-and-tracing-in-net-23c3548ab805